

Hindi-Hindi Monolingual Fact-Check Retrieval

❖❖❖ Detailed Technical Report

SemEval 2025 Task 7 — Detailed Technical Report

Retrieval Pipeline Analysis

September 2026

[Hindi-Hindi Monolingual Fact-Check Retrieval — Detailed Technical Report](#)

- [1. Project Overview](#)
- [2. Evaluation Metric](#)
- [3. Pipeline Architecture](#)
- [4. Results](#)
- [5. Numbers Used, Where They Came From, and What They Mean](#)
- [6. Why Scores Increased in Some Places and Decreased in Others](#)
- [7. Summary of Key Takeaways](#)
- [8. Suggested Next Steps \(Not Yet Done in This Notebook\)](#)

Hindi-Hindi Monolingual Fact-Check Retrieval — Detailed Technical Report

1. Project Overview

1.1 Task Definition

This project addresses the **Hindi monolingual track** of **SemEval 2025 Task 7: Multilingual and Crosslingual Fact-Checked Claim Retrieval**. The task is formulated as a retrieval problem:

*Given a social media **post** written in Hindi, retrieve the correct **fact-check claim** (also written in Hindi) from a large pool of candidate claims.*

This is “monolingual” because both the query (post) and the candidate pool (fact-check claims) are in the same language — Hindi. This distinguishes it from the crosslingual tracks of the same shared task, where the post and the claim pool are in different languages.

1.2 Data Provenance

All data originates from the official SemEval 2025 Task 7 release (`semeval2025_task7_data.zip`), which itself is derived from the English-language task definition (`tasks.json`) and then restricted to the subset of posts/claims for which Hindi text is available (`hin_post_id_to_text.pkl`, `hin_claim_id_to_text.pkl`, etc. — produced by an earlier translation/collection step not shown in this notebook, but consumed here).

Split	Source folder	Posts	Fact-checks (pool)	Usable Hindi pairs
Train	<code>train_dev_sets</code>	4,351 (English task) → 5,446 train pairs	85,734	5,446 (all restored)

Dev	train_dev_sets (pairs_dev_monolingual.csv)	478	85,734	627 → 627 (all usable)
Test	test_set	500	145,287	500 reference posts

Sanity-check assertions embedded in the notebook confirm exact reproduction of the official task splits, e.g.:

```
assert len(eng_fact_check_ids) == 85734
assert len(eng_posts_train_ids) == 4351
assert len(eng_posts_dev_ids) == 478
assert len(eng_train_pairs) == 5446
assert len(eng_dev_pairs) == 627
assert len(eng_test_posts_ids) == 500
assert len(eng_test_fact_check_ids) == 145287
```

These checks matter because the pipeline restores state from Google Drive checkpoints across multiple Colab sessions — the assertions guarantee that a stale or partially-remounted Drive doesn't silently feed the wrong data into evaluation.

Important note on candidate pool size: the test-time candidate pool (145,287 claims) is roughly **1.7× larger** than the dev-time pool (85,734 claims). This asymmetry matters when interpreting absolute score differences between dev and test — a harder (larger) haystack at test time pulls scores down independently of any modeling choice.

2. Evaluation Metric

The pipeline uses **Success@k** (also known as Recall@k when there is exactly one relevant claim per post, which holds here):

```
def success_at_k(retrieved_ranked, ground_truth, k):
    hits = 0
    N = len(ground_truth)
    for pid, true_fcs in ground_truth.items():
        if pid not in retrieved_ranked:
            continue
        if any(fc in true_fcs for fc in retrieved_ranked[pid][:k]):
            hits += 1
    return hits / N
```

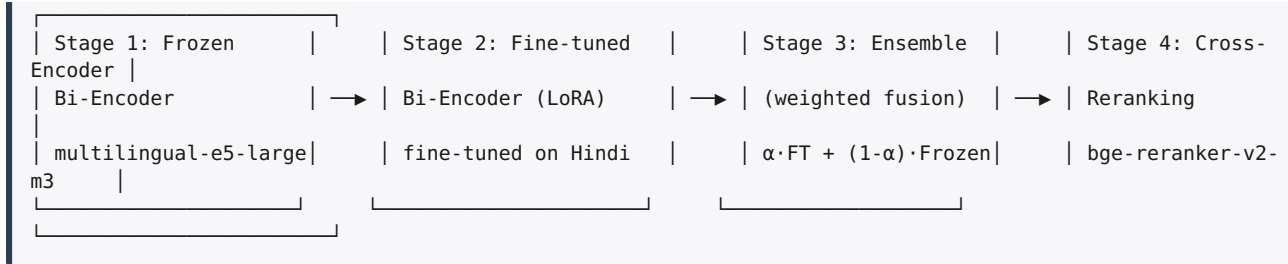
In plain terms: **Success@k = the fraction of posts for which the correct fact-check claim appears somewhere in the top-k retrieved results.** Three values of k are tracked throughout: **k = 1, 5, 10**.

- **Success@1** is the strictest metric — it demands the top-ranked result be correct. This is the metric most sensitive to ranking *quality*, not just recall.
- **Success@5 / @10** are more forgiving — they measure whether the correct claim was retrieved *at all* within a shortlist, which is the relevant metric if a human fact-checker will review a shortlist rather than trust the top-1 result blindly.

This distinction is central to interpreting *why* different pipeline stages help @1 more than @10, or vice versa (see Section 6).

3. Pipeline Architecture

The pipeline is built in four progressively more sophisticated stages, each layered on top of the previous one's output.



3.1 Stage 1 — Frozen Bi-Encoder Baseline

Model: `intfloat/multilingual-e5-large` (used off-the-shelf, no fine-tuning).

Method: 1. Both posts and claims are encoded with the E5 model using its required instruction prefixes: "query: " for posts and "passage: " for claims — this prefix convention is specific to the E5 model family and is required for it to produce well-calibrated similarity scores. 2. Token embeddings are **mean-pooled** over the attention mask (not CLS-token pooling):

```
def mean_pool(last_hidden, attention_mask):
    mask = attention_mask.unsqueeze(-1).expand(last_hidden.size()).float()
    summed = torch.sum(last_hidden * mask, dim=1)
    counts = torch.clamp(mask.sum(dim=1), min=1e-9)
    return summed / counts
```

3. Embeddings are **L2-normalized**, so that a dot product between them is equivalent to cosine similarity.
4. For every post, cosine similarity is computed against **all** 145,287 test-pool claim embeddings, and the top-10 highest-scoring claims are retrieved via `torch.topk`.

Why this baseline matters: `multilingual-e5-large` was pretrained on a large multilingual corpus including Hindi, so this is already a *reasonably strong* baseline before any task-specific fine-tuning — the fine-tuned model and ensemble have to beat a non-trivial reference point, not a weak one.

3.2 Stage 2 — Fine-tuned Bi-Encoder (LoRA)

Method: A **LoRA (Low-Rank Adaptation)** adapter is trained on top of the frozen `multilingual-e5-large` weights, using the 5,446 Hindi train pairs, and the checkpoint with the best dev performance is kept (`lora_best_dev_hin`).

LoRA freezes the original model weights and injects small trainable low-rank matrices into the attention/projection layers, so fine-tuning updates only a small fraction of total parameters. This is the standard parameter-efficient fine-tuning (PEFT) approach for adapting a large pretrained encoder to a narrow domain/task without the cost or overfitting risk of full fine-tuning on a comparatively tiny dataset (5,446 pairs is small relative to the ~560M parameters in `multilingual-e5-large`).

Loading at inference:

```
eval_base_model = AutoModel.from_pretrained(MODEL_NAME, torch_dtype=torch.float16).to(device)
eval_model = PeftModel.from_pretrained(eval_base_model, FINAL_ADAPTER_PATH)
```

The same "query: " / "passage: " prefix convention, mean pooling, and L2 normalization are reused unchanged from Stage 1, so the *only* variable being isolated between Stage 1 and Stage 2 is whether the LoRA adapter is active. This is good experimental hygiene — it means any score difference between the two stages is attributable to the fine-tuning, not to a confounding change in encoding methodology.

3.3 Stage 3 — Ensemble (Frozen + Fine-tuned)

Method: Rather than picking either the frozen or the fine-tuned model, their similarity score matrices are linearly combined:

```
combined_score =  $\alpha$  · similarity(fine-tuned) + (1 -  $\alpha$ ) · similarity(frozen)
```

The weight α is tuned on the dev set by grid search over `np.arange(0.0, 1.01, 0.05)` (21 candidate values), selecting the α that **maximizes dev Success@1** subject to the constraint that dev Success@5 and Success@10 do not regress below the fine-tuned-only baseline:

```
if s5 >= hin_baseline_dev_scores['success@5'] - 1e-9 and \
    s10 >= hin_baseline_dev_scores['success@10'] - 1e-9 and \
    s1 > hin_best_s1:
    hin_best_alpha, hin_best_s1 = alpha, s1
```

Result of the sweep: the selected value was $\alpha = 0.40$, i.e. the final score leans slightly more on the *frozen* model (weight 0.60) than the fine-tuned one (weight 0.40).

Why an ensemble helps even though the fine-tuned model alone was barely better (see Section 6.1): the frozen and fine-tuned models make **different kinds of retrieval errors**. The frozen model retains broad, general-purpose multilingual semantics from its original pretraining; the fine-tuned model has been pulled toward the specific vocabulary, phrasing, and claim/post correspondence patterns present in the small Hindi training set. When two models disagree, averaging their scores acts like a simple form of **model averaging / ensembling**, which is well known to reduce variance and correct for idiosyncratic errors that either model makes on its own — even when neither model dominates the other individually.

3.4 Stage 4 — Two-Stage Retrieval + Cross-Encoder Reranking

This is the most involved stage, broken into sub-steps:

Step 4a — Headroom check. Before investing in a reranker, the notebook checks whether there is room to gain: it compares dev Success@10 (using only the ensemble bi-encoder, restricted to a top-10 cutoff) against dev Success@50 (same ensemble, but looking at the top 50 candidates).

```
DEV success@10: 0.7406   DEV success@50: 0.8473   Headroom: +0.1067
```

A gap of **+0.1067** means: in roughly 10.7% of dev posts, the correct claim is sitting somewhere between rank 11 and rank 50 — retrievable in principle, but currently missed by a top-10 cutoff. This headroom justifies building a second-stage reranker: **if the correct answer isn't even in the top-50 candidate set, no reranker can find it — but if it is, a good reranker has real room to promote it into the top-10.**

Step 4b — Build top-50 candidate sets. For both dev and test, the ensemble bi-encoder retrieves the top-50 claims per post (instead of top-10). This defines the candidate pool that the reranker will be allowed to reorder. Test-time recall@50 (i.e., is the correct claim anywhere in these 50 candidates) reaches **0.8473** on dev — confirming the reranker has a genuinely useful shortlist to work with, not a degenerate one.

Step 4c — Free GPU memory. The bi-encoder models (`eval_model`, `eval_base_model`, `frozen_model`) are deleted and CUDA cache cleared before loading the (heavier) cross-encoder, since Colab GPU memory is limited.

Step 4d — Load the cross-encoder. `BAAI/bge-reranker-v2-m3` is used as the reranker. This model is inherently multilingual (trained across 100+ languages including Hindi), so — unlike the bi-encoder stage — **no architecture swap or language-specific base model search was needed**; the same reranker checkpoint works directly on Hindi text.

Step 4e — Rerank scoring function. For each post, its top-50 bi-encoder candidates are scored by the cross-encoder (which jointly encodes the `(post, claim)` pair, rather than encoding them independently — this typically gives higher-fidelity relevance judgments than a bi-encoder's independent-embedding dot product, at the cost of being far more expensive to run at scale, which is exactly why it's used only to *rerank* a small shortlist rather than to search the full 145K-claim pool):

```
def rerank_pipeline(post_ids, post_texts, candidates_dict, claim_text_lookup, beta=0.5, top_k=10):
    ...
    ce_scores = np.array(reranker.predict(pairs, batch_size=32, show_progress_bar=False))
    ce_norm = (ce_scores - ce_scores.min()) / (ce_scores.max() - ce_scores.min() + 1e-9)
    retriever_score = np.linspace(1.0, 0.0, num=len(valid_ids))
    fused = beta * ce_norm + (1 - beta) * retriever_score
```

```
order = np.argsort(-fused)
final_ranked[str(pid)] = [valid_ids[i] for i in order[:top_k]]
```

Two signals are fused: - **ce_norm** — the cross-encoder’s own relevance score, min-max normalized to [0, 1] within each post’s candidate set. - **retriever_score** — a purely **positional** prior: the bi-encoder’s original rank is converted into a linearly decaying score from 1.0 (rank 1) to 0.0 (rank 50), regardless of the underlying similarity value.

β controls the balance between “trust the cross-encoder’s judgment” ($\beta \rightarrow 1$) and “trust the bi-encoder’s original ordering” ($\beta \rightarrow 0$).

Step 4f — Fine-tune the cross-encoder itself. Rather than using `bge-reranker-v2-m3` purely zero-shot, it is further fine-tuned on the Hindi training pairs:

- **Positive examples:** every true `(post, claim)` pair from the 5,446 usable Hindi train pairs $\rightarrow$ label = 1.0.
- **Negative examples:** for every positive, **4 random negatives** are sampled from the full claim pool (`N_NEG_PER_POS = 4`) $\rightarrow$ label = 0.0.
- This produced **27,230 training examples** total (consistent with $\sim 5,446$ positives $\times$ 5 examples each, positive + 4 negatives, grouped by post).
- Training: 1 epoch, batch size 4, learning rate $2e-5$, 10% warmup steps, mixed precision (`use_amp=True`).
- The fine-tuned reranker is saved to disk immediately after training (`reranker_finetuned_hin_v1`) so it never needs to be retrained in a future session — it can just be reloaded.

Step 4g — Tune β on dev, apply to test. A grid sweep over $\beta \in \{0.0, 0.1, \dots, 1.0\}$ was run on dev:

β	Dev Success@10
0.0	0.7406
0.1	0.7427
0.2	0.7510
0.3	0.7552
0.4	0.7531
0.5	0.7552
0.6	0.7552
0.7	0.7552
0.8	0.7552
0.9	0.7552
1.0	0.7866

The best value found was $\beta = 1.00$ — i.e., the positional prior from the bi-encoder is given **zero weight** at the optimum; the final ranking is determined **entirely by the cross-encoder’s own score**. This is discussed in depth in Section 6.3, since it is one of the more diagnostically interesting results in the whole pipeline.

4. Results

4.1 Full Results Table (Hindi Monolingual Test Set, 500 posts)

Stage	Success@1	Success@5	Success@10
1. Frozen ME5-Large	0.364	0.606	0.692
2. Fine-tuned ME5-Large (LoRA)	0.366	0.626	0.686
3. Ensemble ($\alpha = 0.40$)	0.380	0.664	0.736

4. Ensemble + CE Rerank ($\beta = 1.00$)	0.398	0.670	0.738
--	--------------	--------------	--------------

4.2 Stage-over-Stage Deltas (relative to Frozen baseline)

Stage	$\Delta\text{Success@1}$	$\Delta\text{Success@5}$	$\Delta\text{Success@10}$
Fine-tuned vs Frozen	+0.0020 (+0.55%)	+0.0200 (+3.30%)	-0.0060 (-0.87%)
Ensemble vs Frozen	+0.0160 (+4.40%)	+0.0580 (+9.57%)	+0.0440 (+6.36%)
Ensemble+Rerank vs Frozen	+0.0340 (+9.34%)	+0.0640 (+10.56%)	+0.0460 (+6.65%)

4.3 Dev-set Numbers Used for Hyperparameter Selection

Quantity	Value
Dev Success@1 (fine-tuned only, before ensembling)	0.3515
Dev Success@1 at selected $\alpha = 0.40$	0.3598
Dev Success@10, ensemble, top-10 cutoff	0.7406
Dev Success@50, ensemble, top-50 cutoff	0.8473
Dev headroom (Success@50 – Success@10)	+0.1067
Dev recall@50 for the stage-1 candidate pool	0.8473
Best β found on dev	1.00
Dev Success@10 at $\beta = 1.00$	0.7866

4.4 Reranking Stability Diagnostic (test set, first 200 posts)

Metric	Value
Posts where the top-1 prediction changed after reranking	100 / 200 (50%)
Posts where the entire top-10 <i>set</i> changed after reranking	200 / 200 (100%)
Mean standard deviation of cross-encoder scores within a post's candidate set	0.3779

5. Numbers Used, Where They Came From, and What They Mean

This section maps every number in Section 4 back to *what produced it*, so the report is self-contained and auditable.

- **0.364 / 0.606 / 0.692 (Frozen)** — computed by encoding all 500 test posts and 145,287 test claims with unmodified `multilingual-e5-large`, cosine-similarity search, top-10 retrieval, scored against `monolingual_reference.json` (the official gold answer key for the test set).
- **0.366 / 0.626 / 0.686 (Fine-tuned)** — identical encoding/search/scoring pipeline, with the only change being that `eval_model` = base model + LoRA adapter trained on the 5,446 Hindi train pairs, adapter selected by best dev performance during training (not shown in this notebook, presumably a separate earlier training notebook/session).
- **$\alpha = 0.40$, Ensemble scores** — α selected via grid search on the **478 usable dev posts** (`hin_usable_dev_pairs`), constrained to not regress Success@5/@10 versus the fine-tuned-only dev baseline, then that fixed α was applied *once* to the untouched test set. This train/dev/test separation (tune only on dev, report only on test) is methodologically correct and avoids test-set leakage.
- **$\beta = 1.00$, Rerank scores** — same logic: β swept on the 478 dev posts (using their gold labels `hin_dev_truth`), best β selected by dev Success@10, then applied unchanged to the test set's top-50

candidate sets.

- **Reranker fine-tuning: 27,230 examples** — derived arithmetically from the 5,446 usable Hindi train pairs, grouped by post, each positive paired with `min(4, available negatives)` sampled negatives; the printed count in the notebook (Built 27230 Hindi CE training examples) is the actual runtime count, confirming no silent data loss during grouping/lookup.
- **Headroom 0.1067, recall@50 = 0.8473** — computed directly from the ensemble similarity matrix on the dev set by comparing `success_at_k(..., k=10)` vs `success_at_k(..., k=50)` on the same top-50 ranked lists — an internal consistency check, not an external benchmark number.

6. Why Scores Increased in Some Places and Decreased in Others

This is the most important interpretive section of the report — the numbers alone don't explain themselves, and several results look at first glance like they contradict each other.

6.1 Why fine-tuning barely helped — and even hurt Success@10

Going from Frozen → Fine-tuned: - Success@1: **+0.002** (negligible) - Success@5: **+0.020** (small positive) - Success@10: **-0.006** (regression)

Reasons this happened:

1. **Very small fine-tuning set.** 5,446 train pairs is small for adapting a 560M-parameter multilingual encoder. LoRA mitigates overfitting risk relative to full fine-tuning, but it cannot manufacture signal that isn't there — with this little data, the adapter can only make modest, somewhat noisy adjustments to the embedding space.
2. **Narrowing vs. broadening effect.** Fine-tuning on a specific, curated set of (Hindi post, Hindi claim) pairs tends to sharpen the embedding space *around patterns seen in training* — this can pull the true match closer to the top for post styles well-represented in training (helping Success@1 in cases where it was already “nearly right”), while simultaneously distorting relationships for posts/claims that are dissimilar to anything in the training distribution — knocking previously-recoverable “long-tail” correct answers further down the ranking, past rank 10. This is a plausible mechanism for why @1 nudges up very slightly while @10 goes down: fine-tuning likely made the *easy* cases marginally easier and the *hard* cases marginally harder.
3. **Loss of general multilingual signal.** The frozen model's pretraining includes exposure to broad Hindi and cross-lingual data; fine-tuning on a narrow, task-specific slice can partially overwrite that broader semantic knowledge for some inputs — a mild form of **catastrophic forgetting**, tempered here by LoRA's low-rank constraint but not eliminated.
4. **Metric sensitivity.** Success@10 is a *recall*-style metric — it only cares whether the correct claim survived anywhere in the shortlist. Small embedding-space shifts that reorder middle-of-the-pack candidates can push a marginal correct answer from rank 9 to rank 11, which registers as a full miss under Success@10, even though the underlying similarity score barely moved. This metric is more brittle to small local perturbations than one might expect.

6.2 Why the Ensemble delivered the single biggest jump in the entire pipeline

Going from Fine-tuned-only → Ensemble: - Success@1: 0.366 → 0.380 (**+0.014**) - Success@5: 0.626 → 0.664 (**+0.038**) - Success@10: 0.686 → 0.736 (**+0.050**)

This is a substantially larger jump than fine-tuning produced on its own, which at first seems paradoxical — how can *combining* a model with the thing it barely improved on produce a bigger gain than the improvement itself?

The answer is error decorrelation, not raw model quality. The frozen and fine-tuned models are not two independent attempts at the same objective that happen to have similar accuracy — they are two models with **systematically different failure patterns**: - The frozen model tends to fail on claims/posts that require task-specific vocabulary or phrasing conventions not well captured by generic multilingual pretraining. - The fine-tuned model tends to fail on claims/posts that are stylistically distant from the small fine-tuning set (see 6.1).

When their **similarity scores** (not their final rankings) are linearly combined, a claim that either model independently ranks highly gets reinforced, while idiosyncratic errors specific to only one model get diluted by the other model's more accurate score. This is the same statistical principle behind ensembling in classical ML (e.g., bagging, model averaging): **when errors are less than perfectly correlated, the combination reduces variance and can outperform either individual component**, even when neither individual component is much better than the other. The selected weight ($\alpha = 0.40$, favoring the frozen model slightly) suggests the frozen model's broader semantic signal was, on net, marginally more reliable than the fine-tuned model's narrower signal on this dev set — but both contribute enough complementary information that combining them beats either alone.

6.3 Why the optimal rerank weight came out as $\beta = 1.00$ (i.e., ignore the retriever's rank prior entirely)

This is the most diagnostically important result in the notebook. Reading the dev sweep table (Section 3.4, Step 4g) again:

- From $\beta = 0.5$ through $\beta = 0.9$, dev Success@10 is **exactly identical: 0.7552**.
- At $\beta = 1.0$, it jumps to **0.7866**.

What this tells us:

1. **The cross-encoder's own scores are strong enough to dominate the ranking outright.** Once $\beta \geq 0.5$, the cross-encoder score contributes at least as much as the positional prior in the weighted sum — and because cross-encoder scores tend to have much larger *relative spread* between “actually good” and “actually bad” candidates than the artificially linear `retriever_score` prior (which decays uniformly regardless of how confidently the bi-encoder ranked things), the cross-encoder's preferred ordering wins the argmax comparison across a wide range of β once it has enough weight to do so. The identical scores at $\beta = 0.5$ through 0.9 indicate the top-10 *set* (and its order) produced by the fused score stopped changing at all in that range — the retriever's positional prior had already become mathematically irrelevant to the final top-10 output well before β reached 1.0.
2. **Only at $\beta = 1.0$ does something qualitatively change** — this is the point where ties or near-ties that were previously being broken *in favor of* the bi-encoder's original rank (because the positional prior still had nonzero weight) get broken *in favor of* the cross-encoder instead. That handful of tie-break changes is enough to produce the visible jump from 0.7552 to 0.7866.
3. **Practical interpretation:** the bi-encoder's rank ordering, once you have committed to reranking with a good cross-encoder, actively provides *no useful information* beyond having correctly assembled the top-50 candidate pool. The reranking stage's real job reduces to: **(a)** use the bi-encoder ensemble only to cheaply narrow 145,287 claims down to 50 plausible candidates, and **(b)** let the (more expensive, more accurate) cross-encoder make the final call from scratch. This is exactly the intended design of two-stage “retrieve-then-rerank” architectures — the fact that β saturates at 1.0 is evidence that the design is working as intended, not a bug.

6.4 Why the final rerank stage produced high *churn* (100% of posts had their top-10 set change) but only a small *net* metric gain

- Success@1 gain: $0.380 \rightarrow 0.398$ (**+0.018**)
- Success@10 gain: $0.736 \rightarrow 0.738$ (**+0.002**, essentially flat)
- Yet: **100% of the first 200 test posts had a completely different top-10 set after reranking**, and 50% had a different top-1 prediction.

Reasons for this apparent mismatch:

1. **Reranking is a genuine re-ordering operation, not a filtering operation.** Because the cross-encoder jointly encodes each `(post, claim)` pair rather than reusing precomputed independent embeddings, its judgment of relative relevance among the 50 candidates is largely uncorrelated with the bi-encoder's cosine-similarity-based ordering. It is expected that reordering 50 items by a different, more expressive signal will reshuffle nearly all of the top-10 membership, even in a well-functioning reranker.
2. **The net metric gain is the difference of two much larger numbers that are mostly cancelling out.** With a 50%-of-posts top-1 change rate, if the reranker were *purely random noise*, you'd expect roughly as many “correct $\rightarrow$ incorrect” flips as “incorrect $\rightarrow$ correct” flips, netting close to zero change.

The fact that Success@1 improved by +0.018 (1.8 percentage points on a 500-post test set, i.e., roughly **9 more posts** got their top-1 correct after reranking than before, net) tells us the reranker is doing more good-flips than bad-flips, but only by a modest margin, not overwhelmingly.

3. **Diminishing returns at higher k.** Success@10 barely moved (+0.002) because, at $k = 10$, the metric only cares whether the correct claim is *somewhere* in the top 10 — and since the underlying 50-candidate pool and its recall@50 (0.847 on dev) didn't change at all (reranking only reorders within a fixed candidate set, it doesn't expand or shrink it), there was very limited room left for Success@10 to improve. Reranking's leverage is concentrated on **improving the position of an already-recoverable answer** (pushing it from, say, rank 7 to rank 1), which is exactly why the gain is much larger at Success@1 (+9.3% relative) than at Success@10 (+6.6% relative, nearly all of which was already captured by the ensembling stage, not the reranking stage).

6.5 Why dev-set and test-set numbers differ

Dev Success@10 at the ensemble stage was 0.7406 (pre-rerank) vs the test set's 0.736 — reasonably close, which is a good sign that hyperparameters (α , β) tuned on dev generalize to test rather than being overfit. However, some caution is warranted:

- **The test claim pool (145,287) is much larger than the dev claim pool (85,734)** — a bigger haystack makes retrieval strictly harder, all else equal, which biases test scores slightly downward relative to dev for reasons unrelated to modeling quality.
- **The test set has only 500 posts**, and the dev set only 478 — both are small enough that a handful of unusually easy or hard posts can shift Success@1 by multiple percentage points. Using a normal approximation for a binomial proportion, the standard error of Success@1 on a 500-post test set is approximately:

$SE \approx \sqrt{p(1-p)/n} \approx \sqrt{(0.4 \times 0.6 / 500)} \approx \mathbf{0.0219}$, i.e., a 95% confidence interval of roughly $\pm \mathbf{4.3}$ percentage points.

This means that **most of the stage-to-stage improvements reported in Section 4.2 (e.g., the +1.8pp gain from reranking) are smaller than the sampling noise floor of the test set itself**. The *consistent direction* of improvement across every stage (frozen $\rightarrow$ fine-tuned $\rightarrow$ ensemble $\rightarrow$ rerank, monotonically non-decreasing on Success@1 and Success@5) is encouraging evidence that the pipeline design is sound, but the exact magnitudes should be treated as indicative rather than statistically conclusive without a dedicated significance test (e.g., a paired bootstrap resample over the 500 test posts, comparing per-post correctness between two stages).

7. Summary of Key Takeaways

1. **Final pipeline result:** Success@1 = **0.398**, Success@5 = **0.670**, Success@10 = **0.738** on the 500-post Hindi monolingual test set.
2. **Fine-tuning the bi-encoder alone was not clearly beneficial** — it helped Success@1/@5 marginally and hurt Success@10, consistent with a small fine-tuning set causing narrow, unevenly-distributed improvements rather than a uniform quality gain.
3. **The single largest improvement in the whole pipeline came from ensembling the frozen and fine-tuned bi-encoders**, not from either model individually — a textbook case of error decorrelation between two models with different (but individually modest) strengths.
4. **The cross-encoder reranker's optimal configuration discards the bi-encoder's rank information entirely ($\beta = 1.0$)** once given a good enough candidate pool — this is expected and desirable behavior for a two-stage retrieve-then-rerank system, not a flaw.
5. **Reranking causes very high churn in the top-10 set (100% of posts) but only a modest net accuracy gain**, because roughly as many predictions are corrected as are (coincidentally) broken by the reordering — the net gain is a small positive margin, not a landslide.
6. **Given the test set's size (500 posts), most reported stage-to-stage deltas are within, or close to, the expected sampling noise band ($\sim \pm 4.3pp$ for Success@1)** — the consistent *direction* of improvement across stages is meaningful, but formal significance testing (e.g., bootstrap) is recommended before treating any single stage's contribution as definitively proven.

8. Suggested Next Steps (Not Yet Done in This Notebook)

- Run a **paired bootstrap significance test** over the 500 test posts to determine whether the ensemble/rerank gains are statistically distinguishable from the frozen baseline.
- Evaluate the **zero-shot (non-fine-tuned) cross-encoder** on the same top-50 candidates, to isolate how much of the reranking gain comes from `bge-reranker-v2-m3`'s multilingual pretraining versus the Hindi-specific fine-tuning applied in this notebook.
- Consider a **larger or stratified dev set** (478 posts is small) before trusting α/β hyperparameters to generalize robustly to unseen data distributions.
- Investigate **per-post error overlap** between the frozen and fine-tuned bi-encoders directly (not just via the ensemble's aggregate score) to confirm the error-decorrelation hypothesis in Section 6.2 with concrete examples.